NovaTech Industries — Synthetic Data Generator: Part 1

Checkpoint: 2.6, Step 1 of 4 | **Document ID:** NT-DEV-026A | **Version:** 1.0

Date: 21 October 2026 | **Status:** Implementation exercise

Prerequisite: Python 3.12, VS Code, project virtual environment

Case study: Fictional company and data

1. Objective

Create the directory structure and a minimal, reproducible Python generator for the first three linked datasets: suppliers, products, and supplier-product offerings. This step deliberately avoids quotations, exchange rates, scoring, and approvals, whose policies remain under review.

2. What you will learn

• **CSV:** Comma-separated values; a plain-text table format with one header row.

• **Dataset grain:** What one row represents. For suppliers.csv, one row represents one supplier.

• **Primary key:** Unique identifier for a record (supplier_id).

• **Foreign key:** Identifier referring to another dataset (supplier_id in supplier_products.csv).

• **Reproducibility:** Running the same generator produces identical records.

• **Relative path:** File location constructed relative to the script, not the terminal's working directory.

3. Target folder structure

```
ai-procurement-platform/
■■■ docs/
■   ■■■ data/
■■■ data/
■   ■■■ synthetic/
■       ■■■ v1/
■■■ src/
    ■■■ data_generation/
        ■■■ generate_reference_data.py
```

Run from the project root in VS Code's integrated terminal:

```
mkdir -p src/data_generation data/synthetic/v1
```

4. Create the Python script

Create src/data_generation/generate_reference_data.py with the following code:

```
from pathlib import Path
import csv

PROJECT_ROOT = Path(__file__).resolve().parents[2]
OUTPUT_DIR = PROJECT_ROOT / "data" / "synthetic" / "v1"
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

suppliers = [
    {"supplier_id": "SUP001", "name": "EuroParts GmbH", "country": "Germany", "status": "active"},
    {"supplier_id": "SUP002", "name": "Shenzhen Components", "country": "China", "status": "active"},
    {"supplier_id": "SUP003", "name": "Nordic Industrial AB", "country": "Sweden", "status": "active"},
]

products = [
    {"product_id": "PRD001", "sku": "SENSOR-100", "name": "Industrial sensor", "category": "sensors"},
    {"product_id": "PRD002", "sku": "CTRL-200", "name": "Control module", "category": "controllers"},
]

supplier_products = [
```

```
    {"supplier_id": "SUP001", "product_id": "PRD001", "min_order_qty": 10, "available_qty": 150},
    {"supplier_id": "SUP002", "product_id": "PRD001", "min_order_qty": 20, "available_qty": 400},
    {"supplier_id": "SUP003", "product_id": "PRD002", "min_order_qty": 5, "available_qty": 75},
]


def write_csv(filename, rows):
    if not rows:
        raise ValueError(f"No records provided for {filename}")
    with (OUTPUT_DIR / filename).open("w", newline="", encoding="utf-8") as file:
        writer = csv.DictWriter(file, fieldnames=list(rows[0].keys()))
        writer.writeheader()
        writer.writerows(rows)
    print(f"Created {filename}: {len(rows)} records")


supplier_ids = {row["supplier_id"] for row in suppliers}
product_ids = {row["product_id"] for row in products}

assert len(supplier_ids) == len(suppliers), "Duplicate supplier ID"
assert len(product_ids) == len(products), "Duplicate product ID"
assert len({(r["supplier_id"], r["product_id"]) for r in supplier_products}) == len(supplier_products),
"Duplicate offering"
assert all(r["supplier_id"] in supplier_ids and r["product_id"] in product_ids for r in supplier_products),
"Broken foreign key"
assert all(r["min_order_qty"] > 0 and r["available_qty"] >= 0 for r in supplier_products), "Invalid quantity"

write_csv("suppliers.csv", suppliers)
write_csv("products.csv", products)
write_csv("supplier_products.csv", supplier_products)
```

5. Execute

From the project root:

```
python3 src/data_generation/generate_reference_data.py
```

Expected terminal output:

```
Created suppliers.csv: 3 records
Created products.csv: 2 records
Created supplier_products.csv: 3 records
```

Open data/synthetic/v1/supplier_products.csv in VS Code and confirm that its supplier IDs and product IDs exist in the corresponding parent files.

6. Why this script is designed this way

Path(__file__).resolve().parents[2] locates the project root from the script location. csv.DictWriter uses dictionary keys as CSV column names. The assert statements perform introductory integrity checks; later we'll replace them with formal validation functions and automated tests. We are starting with 8 records rather than the full 231-record target so every relationship is easy to inspect.

7. Completion checklist

- [ ] Folder structure created under the existing project.
- [ ] Script saved in src/data_generation/.
- [ ] All three CSV files generated.
- [ ] Terminal shows counts of 3, 2, and 3.
- [ ] Foreign-key checks pass.
- [ ] Running the script a second time produces the same records.

8. Knowledge check

1. Why does supplier_products.csv contain both supplier_id and product_id?

2. What error would occur if an offering referenced SUP999?

3. Why is reproducible output important when testing procurement rules?

9. Next checkpoint step

Checkpoint 2.6, Step 2: Replace the hardcoded example rows with deterministic generation for the 12 suppliers, 20 products, and 45 supplier-product offerings defined in the data design. Add a manifest and automated integrity tests. Do not proceed until Step 1 is verified.